claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-a04b0e9965c4bdf90.jsonl`
- SHA-256: `0fa3fb1d64693b7fae65d0dbc9c285364e2ab38265c4449b00a3138ddf9c10e0`
- Source modified: `2026-06-20T09:02:23+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T09:00:58.224Z)

```
Adversarially verify this review finding against the ACTUAL code in the working tree. Default to
'false-positive' unless you can point to the concrete line(s) and a real failure scenario.
Context: Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off
origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the
worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py  (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py           (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py   (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py         (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py       (_build_native_runner override; WASB
substrate only)
- backend/config/models.py                            (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py                          (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

FINDING [worker-extract] (major) @ backend/worker/__main__.py:232-234 (cmd_train detector branch
? video_by_stem build) and :244 (video lookup): video_by_stem is built from
`storage.list_uris(f"{corpus}/videos/")` which (LocalStorage.list / object-store list) recurses
and returns EVERY file under videos/, with NO extension filter ? unlike the labels side which
filters to .csv (line 217). Any non-video sidecar (v1.srt, v1.json, v1.txt, a stray thumbnail)
```

is indexed by stem, and on a stem collision the dict silently keeps the LAST-listed entry.
WHY: Confirmed: for files [v1.json, v1.mp4, v1.srt] sorted (the order LocalStorage.list yields), video_by_stem['v1'] ends up = 'v1.srt' ? the real v1.mp4 is shadowed. The runner then runs on a subtitle/sidecar: compute_video_id hashes it, the WASB runner fails (or the stub blindly 'succeeds' on garbage bytes, polluting the training manifest with a trajectory for the wrong file). Buckets routinely carry per-video sidecars, so this is a realistic silent corruption / wrong-skip.
PROPOSED FIX: Filter the videos listing to known video extensions, mirroring the .csv filter: e.g. `if storage.parse_uri(vuri).name.lower().endswith(('.mp4','.mov','.mkv','.avi'))` before indexing; and on collision prefer a video extension / warn. At minimum restrict to the same extensions as security.allowed_upload_extensions.

2. assistant (2026-06-20T09:01:00.061Z)

I'll verify this finding against the actual code in the working tree.

3. user (2026-06-20T09:01:01.215Z)

```
        print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
              f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
        return None
    runner, _ = seg._resolve_runner(config.get("indexing", {}))
    ok, msg = runner.healthcheck()
    if not ok:
        print(f"[worker] detector environment not ready: {msg}")
        return None
    print(f"[worker] detector ready ({args.segmenter}): {msg}")
    return runner


def cmd_train(args: argparse.Namespace) -> int:
    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.

    Two trajectory sources (the train pipeline + harness are identical either way):
      * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
        shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
      * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
        DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
        is the M3.5 "first real training" path; only the trajectory source flips.
    """
    import subprocess

    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
    labels_dir = os.path.join(scratch, "labels")
    traj_dir = os.path.join(scratch, "trajectories")
    videos_dir = os.path.join(scratch, "videos")
    os.makedirs(labels_dir, exist_ok=True)
    os.makedirs(traj_dir, exist_ok=True)

    corpus = args.corpus_uri.rstrip("/")
    label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
                        if u.lower().endswith(".csv"))
    if len(label_uris) < 2:
```

```
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found {len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g. GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir, storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps, frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got {len(specs)}.")
            return 2
        manifest_path = os.path.join(scratch, "manifest.json")
        with open(manifest_path, "w", encoding="utf-8") as f:
            json.dump(specs, f, indent=2)
        src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
```

```
stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg)
                       if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0
```

4. assistant (2026-06-20T09:01:05.035Z)

I have the core code. Now let me verify the claims about `storage.list_uris`, `parse_uri`, and `LocalStorage.list` ordering behavior, plus the `.csv` filter and extension filters.

5. user (2026-06-20T09:01:06.000Z)

No files found

6. user (2026-06-20T09:01:06.317Z)

```
Found 8 files
backend\storage\s3.py
backend\storage\ref.py
backend\storage\gdrive.py
backend\storage\local.py
backend\storage\gcs.py
backend\storage\__init__.py
backend\storage\base.py
docs\PLATFORM_ARCHITECTURE.md
```

7. user (2026-06-20T09:01:09.626Z)

```
"""Pluggable storage layer: local (default) + S3/GCS first-class buckets + Drive (follow-up).

URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are imported
lazily, only when a non-local URI is actually used (install the ``storage-s3`` / ``storage-gcs``
/ ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps and
preserves today's behaviour byte-for-byte.

```

```python
Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an identity
passthrough. See ``docs/DECOUPLED_COMPUTE/``.
"""
from __future__ import annotations

import os
from typing import Any, Dict, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat, parse_uri

__all__ = [
    "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
    "get_storage", "fetch", "put", "list_uris",
]


def _backend_class(name: str):
    """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
    if name == "local":
        from backend.storage.local import LocalStorage
        return LocalStorage
    if name == "s3":
        from backend.storage.s3 import S3Storage
        return S3Storage
    if name == "gcs":
        from backend.storage.gcs import GCSStorage
        return GCSStorage
    if name == "gdrive":
        from backend.storage.gdrive import GDriveStorage
        return GDriveStorage
    raise ValueError(f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive)")


def get_storage(backend: Optional[str] = None, *,
                config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
    """Construct a backend. ``backend`` (or ``config['backend']``, default 'local') selects
    the implementation; its per-backend settings come from ``config[backend]`` merged with
    ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-selectors."""
    cfg = config or {}
    name = backend or cfg.get("backend", "local")
    settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
    settings.update(overrides)
    return _backend_class(name)(**settings)


def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
          config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
    """Resolve ``uri`` to a local file path. For ``local`` this is an identity passthrough
    (no copy); for s3/gcs it downloads to ``dst``."""
    ref = parse_uri(uri)
    return get_storage(ref.backend, config=config).fetch_to_local(ref, dst, overwrite=overwrite)


def put(local_path: "str | os.PathLike[str]", uri: str, *,
        config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) -> str:
```

```
    """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
    ref = parse_uri(uri)
    get_storage(ref.backend, config=config).put(local_path, ref, content_type=content_type)
    return uri


def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) -> Iterator[str]:
    """Yield the URI of every object under ``prefix_uri``."""
    ref = parse_uri(prefix_uri)
    for r in get_storage(ref.backend, config=config).list(ref):
        yield r.uri

```

8. user (2026-06-20T09:01:09.810Z)

```
"""Local-filesystem storage backend ? the default, zero new dependencies.

Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough (the
file is already local), ``put`` is a copy (a no-op when src == dst). With ``backend='local'``
the whole pipeline behaves exactly as before the storage layer existed.
"""
from __future__ import annotations

import os
import shutil
from datetime import datetime
from typing import Iterator

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat


class LocalStorage(StorageBackend):
    def __init__(self, *, base_dir: str = "", **_ignored) -> None:
        # Relative keys resolve under base_dir when set; absolute keys are used as-is.
        self.base_dir = base_dir

    def _path(self, ref: StorageRef) -> str:
        p = ref.key
        if self.base_dir and not os.path.isabs(p):
            return os.path.join(self.base_dir, p)
        return p

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        # The object is already on the local FS ? return its path (identity). ``dst`` is
        # honoured only if it names a *different* location (a copy is then made).
        src = self._path(ref)
        if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
            return src
        dst = str(dst)
        if os.path.exists(dst) and not overwrite:
            return dst
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        shutil.copyfile(src, dst)
        return dst

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        dst = self._path(ref)
        if os.path.abspath(str(local_path)) != os.path.abspath(dst):
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            shutil.copyfile(str(local_path), dst)
```

```
            return ref

    def exists(self, ref) -> bool:
        return os.path.exists(self._path(ref))

    def stat(self, ref) -> StorageStat:
        st = os.stat(self._path(ref))
        return StorageStat(size=st.st_size, modified=datetime.fromtimestamp(st.st_mtime))

    def list(self, prefix) -> Iterator[StorageRef]:
        base = self._path(prefix)
        if os.path.isdir(base):
            for root, _dirs, files in os.walk(base):
                for fn in sorted(files):
                    full = os.path.join(root, fn)
                    yield StorageRef("local", "", full, f"local://{full}")
        elif os.path.exists(base):
            yield StorageRef("local", "", base, f"local://{base}")

    def delete(self, ref) -> None:
        path = self._path(ref)
        if os.path.exists(path):
            os.remove(path)

```

9. assistant (2026-06-20T09:01:13.112Z)

Now I can confirm the key mechanics. `LocalStorage.list` recurses (`os.walk`) and yields ALL files with no extension filter, in sorted order per directory. Let me verify the `parse_uri` `.name` property and check what `compute_video_id` and the runner do, plus confirm the labels side filters `.csv`.

10. user (2026-06-20T09:01:13.693Z)

```
"""Storage value objects + URI parsing for the pluggable storage layer.

A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:

    <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4, /data/x.mp4)
    local://<path>              -> local   (local:///abs or local://rel/p)
    s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via endpoint_url)
    gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
    gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)

No cloud SDKs are imported here ? this module is pure stdlib so ``import backend.storage``
stays cheap.
"""
from __future__ import annotations

import re
from dataclasses import dataclass
from datetime import datetime
from typing import Optional

_SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
_KNOWN = ("local", "s3", "gcs", "gdrive")


@dataclass(frozen=True)
class StorageStat:
    """Lightweight object metadata (uniform across backends)."""
```

```
    size: int
    modified: Optional[datetime] = None
    etag: Optional[str] = None
    content_type: Optional[str] = None


@dataclass(frozen=True)
class StorageRef:
    """A backend + a location. ``bucket`` is the S3/GCS bucket or the Drive folder-id;
    for ``local`` it is empty and ``key`` carries the filesystem path."""
    backend: str
    bucket: str
    key: str
    uri: str

    @property
    def name(self) -> str:
        return self.key.rstrip("/").rsplit("/", 1)[-1]


def parse_uri(uri: str) -> StorageRef:
    """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.

    A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
    have no ``//``) is treated as a local filesystem path ? preserving today's behaviour.
    """
    s = str(uri)
    m = _SCHEME_RE.match(s)
    if not m:
        return StorageRef("local", "", s, f"local://{s}")
    scheme = m.group(1).lower()
    rest = s[m.end():]
    if scheme == "gs":
        scheme = "gcs"
    if scheme == "local":
        return StorageRef("local", "", rest, s)
    if scheme in _KNOWN:
        bucket, _, key = rest.partition("/")
        return StorageRef(scheme, bucket, key, s)
    raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")

```

11. assistant (2026-06-20T09:01:16.183Z)

Now let me check the S3 storage `list` to confirm cross-backend behavior, and verify the runner's `run_predict` to understand what happens when fed a non-video file.

12. user (2026-06-20T09:01:16.983Z)

```
"""S3 (and S3-compatible) storage backend ? first-class object buckets.

One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they differ
only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily so
``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.

Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` / ``AWS_SECRET_ACCESS_KEY``
/ ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
(tests pass a fake; production lets the constructor build one via boto3).
"""
from __future__ import annotations
```

```
import os
from typing import Any, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat


class S3Storage(StorageBackend):
    def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
                 region: Optional[str] = None, addressing_style: str = "auto",
                 signed_url_ttl: int = 3600, **_ignored) -> None:
        self._ttl = signed_url_ttl
        if client is not None:
            self._client = client
            return
        import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
        from botocore.config import Config  # type: ignore
        self._client = boto3.client(
            "s3",
            endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
            region_name=region,
            config=Config(s3={"addressing_style": addressing_style}),
        )

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        if dst is None:
            raise ValueError("S3Storage.fetch_to_local requires a dst path")
        dst = str(dst)
        if os.path.exists(dst) and not overwrite:
            return dst
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        self._client.download_file(ref.bucket, ref.key, dst)
        return dst

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        extra = {"ContentType": content_type} if content_type else None
        self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
        return ref

    def exists(self, ref) -> bool:
        try:
            self._client.head_object(Bucket=ref.bucket, Key=ref.key)
            return True
        except Exception:  # noqa: BLE001 - any boto/client error means "not found / unreachable"
            return False

    def stat(self, ref) -> StorageStat:
        h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
        return StorageStat(size=int(h.get("ContentLength", 0)), modified=h.get("LastModified"),
                           etag=h.get("ETag"), content_type=h.get("ContentType"))

    def list(self, prefix) -> Iterator[StorageRef]:
        paginator = self._client.get_paginator("list_objects_v2")
        for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
            for obj in page.get("Contents", []) or []:
                key = obj["Key"]
                yield StorageRef("s3", prefix.bucket, key, f"s3://{prefix.bucket}/{key}")

    def delete(self, ref) -> None:
        self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
```

```
    def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
        op = "get_object" if method.upper() == "GET" else "put_object"
        return self._client.generate_presigned_url(
            op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or self._ttl)

```

13. assistant (2026-06-20T09:01:20.389Z)

S3 `list` confirms: it returns ALL keys under the prefix with no extension filter, ordered as the bucket returns them (lexicographic for S3). Now let me examine the native runner's `run_predict` to understand the consequence of being handed a non-video file, and check the stub runner behavior.

14. user (2026-06-20T09:01:20.930Z)

```
"""Linux-native WASB shuttle-trajectory runner (no WSL).

The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
"Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md`` M3.5).
It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py`` core
that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
transport:

  * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
    ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
  * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
  * weights / repo / python / scratch come from the **environment** first
    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.

It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
(``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``); ``detector_impl="native"``.

**Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL runner
(Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV and
identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
test suite.
"""
import os
import shutil
import logging
import subprocess
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.detectors.base import DetectorRunner
# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to WSL/TrackNet.
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

logger = logging.getLogger(__name__)

```

```python
# Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
# detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
_INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


@dataclass
class NativeWasbConfig:
    """Resolved settings for a Linux-native WASB run.

    Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
    over ``indexing.wasb.*`` config (the box sets env; no config edits needed). ``device``
    defaults to ``cuda`` (the WSL byte-parity path).
    """
    repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
    python_bin: str = "python"             # the `wasb` conda env python (invoked by path, no activate)
    weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
    sport: str = "badminton"
    device: str = "cuda"                   # cuda | mps | cpu
    scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next to the output dir
    timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no timeout)
    keep_frames: bool = False              # retain the decoded frame cache after success (debug only)
    stream_video: bool = False             # fast path: decode in-process, no PNG extraction (bit-identical)

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
        w = (idx_cfg or {}).get("wasb", {}) or {}
        env = os.environ.get
        return cls(
            repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
            python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
            weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
            sport=w.get("sport", "badminton"),
            device=env("WASB_DEVICE") or w.get("device", "cuda"),
            scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or "",
            timeout_sec=int(w.get("timeout_sec", 1800)),
            keep_frames=bool(w.get("keep_frames", False)),
            stream_video=bool(w.get("stream_video", False)),
        )


class NativeWasbRunner(DetectorRunner):
    """WASB runner that executes natively on a Linux GPU box ? no WSL. See module docstring."""

    def __init__(self, cfg: NativeWasbConfig):
        self.cfg = cfg

    # --- low-level native exec (the single place tests patch) --------------------------
    def _run(self, argv: List[str], cwd: Optional[str] = None) -> subprocess.CompletedProcess:
        logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
        return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
                              timeout=(self.cfg.timeout_sec or None))

    def _repo_src(self) -> str:
        return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
```

```python

    def _copy_infer_script(self) -> bool:
        """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds configs/."""
        repo_src = self._repo_src()
        try:
            os.makedirs(repo_src, exist_ok=True)
            shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
            return True
        except OSError as e:
            logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
            return False

    def _rm_rf(self, path: Optional[str]) -> None:
        """Best-effort recursive delete of a native path (post-success cleanup; never raises)."""
        if path:
            shutil.rmtree(path, ignore_errors=True)

    def healthcheck(self) -> Tuple[bool, str]:
        """Verify weights + repo present and the `wasb` python imports torch (and sees a GPU on cuda)."""
        if not self.cfg.weights_path:
            return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or indexing.wasb.weights_path).")
        weights = os.path.expanduser(self.cfg.weights_path)
        if not os.path.exists(weights):
            return False, f"WASB weights not found at {weights}"
        repo_src = self._repo_src()
        if not os.path.isdir(repo_src):
            return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
                           f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
        code = "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
        try:
            res = self._run([self.cfg.python_bin, "-c", code])
        except FileNotFoundError:
            return False, f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON)."
        except subprocess.TimeoutExpired:
            return False, "native torch healthcheck timed out."
        out = (res.stdout or "").strip()
        if res.returncode != 0:
            return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or out).strip()}"
        if self.cfg.device == "cuda" and "cuda True" not in out:
            return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
        return True, out

    def run_predict(self, video_win_path: str, output_win_dir: str,
                    video_id: Optional[str] = None) -> Optional[str]:
        """Run WASB natively on a video. Returns the path to the trajectory CSV, or None.

        Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the WSL
        runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte the
        same downstream contract, so the parity comparison is apples-to-apples.
        """
        output_dir = os.path.abspath(output_win_dir)
        os.makedirs(output_dir, exist_ok=True)
        # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
        norm = str(video_win_path).replace("\\", "/")
```

```
        stem, _ext = os.path.splitext(os.path.basename(norm))
        # Content-derived key: same filename + different bytes -> different key (no cache reuse).
        key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
        out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

        if not self._copy_infer_script():
            logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
            return None

        weights = os.path.expanduser(self.cfg.weights_path)
        argv = [self.cfg.python_bin, "wasb_infer.py",
                "--video", os.path.abspath(str(video_win_path))]
        frames_dir: Optional[str] = None
        if self.cfg.stream_video:
            # Fast path: decode in-process, no PNG extraction. Output is bit-identical to disk.
            argv.append("--stream-video")
        else:
            scratch = self.cfg.scratch_dir or output_dir
            frames_dir = os.path.join(scratch, f"{key}_frames")
            argv += ["--frames_out_dir", frames_dir]
        argv += ["--weights", weights,
                 "--sport", self.cfg.sport,
                 "--device", self.cfg.device,
                 "--out", out_csv]

        logger.info("Running native WASB inference (device=%s) on %s ...",
                    self.cfg.device, os.path.basename(norm))
        try:
            res = self._run(argv, cwd=self._repo_src())
        except FileNotFoundError:
            logger.error("python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin)
            return None
        except subprocess.TimeoutExpired:
            logger.error("WASB native inference timed out after %ss.", self.cfg.timeout_sec)
            return None
        if res.returncode != 0:
            logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout or "")[-2000:])
            return None
        if not os.path.exists(out_csv):
            logger.error("WASB native finished but expected CSV not found at %s", out_csv)
            return None
        logger.info("WASB trajectory CSV: %s", out_csv)

        # Success ? the CSV is the durable output; the decoded frame cache is a pure intermediate.
        # On any earlier failure we return above WITHOUT cleaning, so the cache survives a resume.
        # (Streaming never materializes frames, so there is nothing to drop on that path.)
        if not self.cfg.keep_frames and frames_dir:
            logger.info("Cache hygiene: removing frame cache for %s "
                        "(set indexing.wasb.keep_frames=true to retain).", key)
            self._rm_rf(frames_dir)
        return out_csv

    # Trajectory parsing + windowing are inherited from tracknet_runner (model-agnostic).
    parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
    trajectory_to_action_windows =
```

```
staticmethod(TrackNetRunner.trajectory_to_action_windows)
207
```

15. assistant (2026-06-20T09:01:27.604Z)

Now I understand the runner fully. When fed a non-video file (e.g. `v1.srt`), `wasb_infer.py` would be invoked with `--video v1.srt`. Let me confirm what would happen ? whether it fails loudly (returncode != 0 ? returns None ? "skipping") or silently succeeds. The key question for the finding's "silent corruption" severity. Let me check the labels-side filter and verify the realistic order of files.

Let me verify the exact claim about ordering. The finding says for `[v1.json, v1.mp4, v1.srt]` sorted, `video_by_stem['v1']` ends up as `v1.srt`. Since `os.walk` yields `sorted(files)` per directory, the iteration order within one dir is `v1.json, v1.mp4, v1.srt`, and the dict keeps the LAST write = `v1.srt`. That's confirmed mechanically.

Let me check whether sidecars actually land in `videos/` in this project's conventions, and whether the worker's infer path (not just train) has the same pattern.

16. user (2026-06-20T09:01:29.644Z)

```
98:    local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
224:    # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
232:        for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
245:                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
319:    pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
326:                         "(needs videos/ in the corpus; honours detector_impl).")
```

17. user (2026-06-20T09:01:30.049Z)

```
backend\api\uploads.py:50:    exts = [e.lower().lstrip(".") for e in (getattr(sec,
"allowed_upload_extensions", None) or ["mp4", "mov"])]
backend\config\models.py:288:    allowed_upload_extensions: List[str] = ["mp4", "mov"]
docs\CODE_MAP.md:115:[Omitted long matching line]
```

18. assistant (2026-06-20T09:01:34.672Z)

So the `infer` path (line 98) uses a single explicit `args.video_uri` ? not affected. The vulnerability is specific to the `cmd_train` detector branch. The project does have an `allowed_upload_extensions` config defaulting to `["mp4", "mov"]`, confirming the codebase's own notion of valid video extensions, and confirming the proposed-fix reference.

Now let me check the test that was added for the worker to see whether it tests the videos/ listing, and whether any existing convention puts sidecars in videos/. Let me also confirm how a corpus's `videos/` is populated.

19. user (2026-06-20T09:01:35.106Z)

```
"""Worker dispatcher: pull (storage) ? run pipeline ? push (storage).

`cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is
validated
on the droplet against MinIO.
"""
import json

import pytest

from backend.config import load_config
from backend.worker import __main__ as wmain

```

```
# ---------------------------------------------------------------- pure helpers

@pytest.mark.parametrize("raw,expected", [
    ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid", "wasb_hybrid"),
])
def test_coerce(raw, expected):
    assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


def test_apply_overrides_on_typed_config():
    cfg = _fresh_config()
    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true", "indexing.min_segment_duration=1.5",
                                 "sport=tennis"])
    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.min_segment_duration == 1.5
    assert cfg.sport == "tennis"


def test_apply_overrides_rejects_bad_format():
    cfg = _fresh_config()
    with pytest.raises(ValueError):
        wmain._apply_overrides(cfg, ["no_equals_sign"])


def test_write_intervals_csv(tmp_path):
    segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence": "LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.5}]
    out = tmp_path / "intervals.csv"
    wmain._write_intervals_csv(segs, str(out))
    lines = out.read_text().splitlines()
    assert lines[0] == "start,end,shot_count,ending_reason"
    assert lines[1].startswith("2.000,5.000,4,")
    assert lines[2].startswith("9.000,12.500,,")


def test_write_report_json(tmp_path):
    out = tmp_path / "report.json"
    wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success", str(out))
    data = json.loads(out.read_text())
    assert data["video_id"] == "vid1" and data["segment_count"] == 1
    assert data["segments"][0] == {"start": 1.0, "end": 2.0}


def test_parser_infer_accepts_set_and_uris():
    args = wmain.build_parser().parse_args([
        "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
        "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
    assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


# ---------------------------------------------------------------- cmd_infer (local, mocked)

class _OKResult:
    passed = True
    validator_name = "fake"
    message = "ok"

    def model_dump(self):
```

```
            return {"validator_name": "fake", "passed": True, "message": "ok"}


class _FailResult(_OKResult):
    passed = False
    message = "too blurry"


class _FakeSeg:
    def __init__(self, db, config):
        pass

    def process_video(self, video_id, path, max_frames=None):
        return [
            {"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence": "LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.0, "shot_count": 6, "shot_count_confidence": "LocalNoAI"},
        ]


def _fresh_config():
    # Load defaults without touching any cwd config.json.
    import tempfile
    p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
    p.write("{}")
    p.close()
    return load_config(p.name)


@pytest.fixture
def mocked_pipeline(monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_OKResult()], {}))
    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
    video = tmp_path / "in.mp4"
    video.write_bytes(b"FAKEVIDEO")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    out = tmp_path / "bucket"
    scratch = tmp_path / "scratch"

    rc = wmain.main([
        "infer", "--video-uri", str(video), "--out-uri", str(out),
        "--config", str(cfg), "--scratch", str(scratch),
        "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
    ])
    assert rc == 0
    intervals = out / "outputs" / "vid123" / "intervals.csv"
    report = out / "outputs" / "vid123" / "report.json"
    assert intervals.exists() and report.exists()
    rows = intervals.read_text().splitlines()
    assert len(rows) == 3                          # header + 2 rallies
    assert json.loads(report.read_text())["segment_count"] == 2


def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_FailResult()], {}))
```

```python
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
    video = tmp_path / "in.mp4"
    video.write_bytes(b"X")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "o"),
                     "--config", str(cfg), "--scratch", str(tmp_path / "s")])
    assert rc == 2


# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

def test_cmd_train_orchestration(tmp_path, monkeypatch):
    import os
    import subprocess

    labels_dir = tmp_path / "corpus" / "labels"
    labels_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri", str(out_dir),
                     "--version", "0.0.1", "--config", str(cfg), "--scratch", str(scratch)])
    assert rc == 0
    # bundle pushed to <out-uri>/weights/<version>/
    assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
    assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
    # the manifest fed to the harness enumerated both labeled videos with a trajectory each
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2
    assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


def test_parser_train_detector_args():
    args = wmain.build_parser().parse_args([
        "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version", "1.0.0",
        "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
    assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"


def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
```

```
    """--trajectory-source detector pulls videos + runs the resolved DetectorRunner. With
    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the runner)."""
    import os
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no GPU/WSL)
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
    (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.2", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2
    # Real-extract path: each spec points at an on-disk trajectory the runner actually wrote.
    assert all(os.path.exists(s["trajectory"]) for s in specs)
    assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub runner's output

```

20. assistant (2026-06-20T09:01:38.638Z)

The test only exercises a clean `videos/` directory with exactly `v1.mp4` and `v2.mp4` ? no sidecars. So the bug is not covered by tests and the happy path masks it.

Now let me examine the stub_runner to understand the "blindly succeeds on garbage bytes" half of the claim, which affects severity (silent corruption vs. loud skip).

21. user (2026-06-20T09:01:39.211Z)

```
"""CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".

A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
CSV with **no GPU, no WSL, no model weights** ? so the whole trajectory-hybrid pipeline
(``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
```

```
(``use_gpu=False``); this closes the last GPU/WSL-only gap so the *entire* pipeline is
runnable ? and regression-gateable ? without a GPU.

Two modes:
  * **replay** ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
    pipeline with a committed/golden trajectory; deterministic + cross-OS).
  * **synth** (default) ? generate a deterministic synthetic trajectory of
    ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
    gaps, so windowing yields a stable, known number of candidate windows.

This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
"Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
(M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
WSL runners stay the default, so production behaviour is unchanged.
"""
import os
import csv as _csv
import shutil
import logging
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.detectors.base import DetectorRunner
# Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

logger = logging.getLogger(__name__)


@dataclass
class StubConfig:
    """Settings for the CPU stub runner (built from ``config.json`` -> ``indexing.stub``)."""
    fps: float = 30.0
    frame_width: float = 1280.0
    num_rallies: int = 2
    rally_seconds: float = 4.0
    gap_seconds: float = 3.0           # dead gap between rallies (> merge_gap so windows don't merge)
    lead_seconds: float = 2.0          # dead lead-in
    step_px: float = 12.0              # per-frame shuttle displacement during a rally (>> velocity_thresh)
    replay_csv: Optional[str] = None   # if set + exists, emit this CSV verbatim instead of synthesising

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
        s = (idx_cfg or {}).get("stub", {}) or {}
        return cls(
            fps=float(s.get("fps", 30.0)),
            frame_width=float(s.get("frame_width", 1280.0)),
            num_rallies=int(s.get("num_rallies", 2)),
            rally_seconds=float(s.get("rally_seconds", 4.0)),
            gap_seconds=float(s.get("gap_seconds", 3.0)),
            lead_seconds=float(s.get("lead_seconds", 2.0)),
            step_px=float(s.get("step_px", 12.0)),
            replay_csv=s.get("replay_csv"),
        )


class StubDetectorRunner(DetectorRunner):
    """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""

    def __init__(self, cfg: Optional[StubConfig] = None):
```

```
        self.cfg = cfg or StubConfig()

    def healthcheck(self) -> Tuple[bool, str]:
        return True, "stub CPU detector (no GPU/WSL)"

    def _rows(self) -> List[Tuple[int, int, float, float]]:
        """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight bursts
        separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame ?
        well above the default 0.02 velocity threshold ? so every rally frame is "active"
        and each burst becomes exactly one candidate window."""
        cfg = self.cfg
        rows: List[Tuple[int, int, float, float]] = []
        frame = 0
        y = cfg.frame_width / 4.0
        base_x = cfg.frame_width / 2.0
        for _ in range(int(cfg.lead_seconds * cfg.fps)):
            rows.append((frame, 0, 0.0, 0.0))
            frame += 1
        for r in range(cfg.num_rallies):
            for i in range(int(cfg.rally_seconds * cfg.fps)):
                x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
                rows.append((frame, 1, x, y))
                frame += 1
            if r != cfg.num_rallies - 1:
                for _ in range(int(cfg.gap_seconds * cfg.fps)):
                    rows.append((frame, 0, 0.0, 0.0))
                    frame += 1
        return rows

    def run_predict(self, video_win_path: str, output_win_dir: str,
                    video_id: Optional[str] = None) -> Optional[str]:
        """Write a deterministic trajectory CSV and return its path (never touches the GPU/WSL)."""
        os.makedirs(output_win_dir, exist_ok=True)
        stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\", "/")))[0]
        key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
        out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")

        if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
            shutil.copyfile(self.cfg.replay_csv, out_csv)
            logger.info("Stub runner: replayed trajectory %s -> %s", self.cfg.replay_csv, out_csv)
            return out_csv

        rows = self._rows()
        with open(out_csv, "w", newline="") as f:
            w = _csv.writer(f)
            w.writerow(["Frame", "Visibility", "X", "Y"])
            for frame, vis, x, y in rows:
                w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
        logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows), out_csv)
        return out_csv

    # Model-agnostic CSV parsing + windowing ? identical to the real runners.
    parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
    trajectory_to_action_windows = staticmethod(TrackNetRunner.trajectory_to_action_windows)


def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
```

```
125                                         frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                         step_px: float = 12.0) -> str:
127         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y trajectory whose
128         in-flight (visible + oscillating) frames cover each labeled rally window and whose dead frames
129         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.
130
131         This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent data with
132         no GPU/WSL, validating the whole training pipeline before the real WASB detector lands. (The
133         learned model is meaningful for plumbing, not for production ? the trajectory is synthetic.)
134         """
135         rallies: List[Tuple[float, float]] = []
136         with open(labels_csv, newline="", encoding="utf-8") as f:
137             for row in _csv.DictReader(f):
138                 try:
139                     rallies.append((float(row["start_time"]), float(row["end_time"])))
140                 except (KeyError, ValueError):
141                     continue
142         duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143         nframes = max(1, int(duration * fps))
144         base_x = frame_width / 2.0
145         y = frame_width / 4.0
146         os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147         with open(out_csv, "w", newline="", encoding="utf-8") as f:
148             w = _csv.writer(f)
149             w.writerow(["Frame", "Visibility", "X", "Y"])
150             for i in range(nframes):
151                 t = i / fps
152                 if any(s <= t < e for s, e in rallies):
153                     x = base_x + (step_px if i % 2 else -step_px)
154                     w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155                 else:
156                     w.writerow([i, 0, "0.000", "0.000"])
157         return out_csv
158
```

22. assistant (2026-06-20T09:01:49.155Z)

This is a critical confirmation of the severity nuance. The `StubDetectorRunner.run_predict` (line 97-117) **completely ignores the video bytes**. It writes a deterministic synthetic trajectory regardless of what `video_win_path` points at. So when `RALLY_DETECTOR_IMPL=stub` is set and a sidecar shadows the real video:
- `compute_video_id` hashes the sidecar (`v1.srt`) ? wrong video_id (wrong cache key)
- The stub "succeeds" ? writes a trajectory keyed to the wrong video_id ? **pollutes the training manifest silently** (exactly as the finding states).

For the native runner on a real GPU box: `wasb_infer.py --video v1.srt` would almost certainly fail (ffmpeg/decoder can't decode a subtitle file) ? returncode != 0 ? `run_predict` returns None ? "detector produced no trajectory ... skipping" ? a real video is silently skipped (wrong-skip). Either path is a real defect.

Now let me confirm the finding's exact line-number claims and the precise labels-side `.csv` filter contrast. The finding cites line 217 (labels filter) and lines 232-234 / 244 (videos build + lookup). My read confirms:
- Line 217-218: labels filtered to `.csv`.
- Lines 232-234: `video_by_stem[stem] = vuri` with NO extension filter.
- Line 243-246: lookup + skip.

Let me also double check `compute_video_id` is the worker-imported one and there isn't a guard I'm missing in `_resolve_train_detector` or elsewhere that filters videos.

23. user (2026-06-20T09:01:50.313Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...        # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""
from __future__ import annotations

import argparse
import csv
import json
import os
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
```

24. assistant (2026-06-20T09:02:07.967Z)

I have everything needed to render a verdict. Let me finalize my assessment.

The finding is mechanically accurate and confirmed against the working tree:

1. **No extension filter on videos** (lines 232-234): `for vuri in storage.list_uris(f"{corpus}/videos/", ...)` then `video_by_stem[os.path.splitext(vname)[0]] = vuri` ? confirmed, no filtering by extension.

2. **Asymmetry with labels** (lines 217-218): labels ARE filtered to `.csv` via `if u.lower().endswith(".csv")`. The contrast is real and the videos side lacks the equivalent guard.

3. **Last-write-wins on stem collision**: confirmed by `LocalStorage.list` (`os.walk` + `sorted(files)`) yielding `v1.json, v1.mp4, v1.srt` in order, so the dict ends with `v1.srt`. S3 `list` similarly returns all keys lexicographically with no filter.

4. **Realistic silent corruption**: With `RALLY_DETECTOR_IMPL=stub` (an explicitly documented/supported wiring mode, used in `test_cmd_train_detector_source_runs_real_extract_via_stub`), `StubDetectorRunner.run_predict` ignores the video bytes entirely and "succeeds" on any path ? so a shadowing `v1.srt` produces a trajectory keyed to the wrong `compute_video_id` and pollutes the training manifest silently. On the native GPU path, `wasb_infer.py --video v1.srt` fails ? trajectory None ? the real video is silently skipped (wrong-skip). Both are real failure modes.

A couple of honest counterpoints to weigh on severity:
- This is a NEW code path (`--trajectory-source detector`), default source stays `synth`, so it is opt-in and not yet on the default e2e.
- It requires sidecars to actually be placed in `videos/` ? plausible per the project's bucket conventions but not proven to currently exist in the corpus.
- The native runner half degrades to a loud-ish skip (logged), not silent corruption; only the stub half is truly silent. That makes "silent corruption" partly true (stub) and partly an overstatement (native logs a skip).

Net: a real, confirmed defect with a correct proposed fix. Severity "major" is slightly high given it is an opt-in, not-yet-default path that on the production (native) runner degrades to a logged skip rather than silent corruption; "minor" understates the stub-path silent manifest pollution. I'll keep it as a legitimate finding but corrected to between ? leaning to keeping major is defensible, but the most honest call is minor-to-major. I'll mark it real with corrected severity "minor" because the truly-silent corruption requires the stub mode, and the default/native path fails loudly-ish; the data-correctness risk is real but gated behind an opt-in flag and atypical bucket contents.

25. user (2026-06-20T09:02:19.314Z)

Structured output provided successfully

26. assistant (2026-06-20T09:02:23.614Z)

Verdict: real (corrected to minor). The finding is mechanically accurate ? confirmed in the working tree across `backend/worker/__main__.py:217,232-234,243-246`, `backend/storage/local.py:56-64`, and `backend/storage/s3.py:65-70`. I downgraded severity from major to minor because the truly-silent corruption only occurs in the opt-in stub mode (`backend/pipeline/detectors/stub_runner.py:97-117` ignores video bytes), while the production native path degrades to a logged skip; the path is also default-OFF (`synth`) and requires sidecars in `videos/`. The proposed fix is correct.